```
>>>
>>> list2 = [each*2 for each in list1]
>>>
```

List2 contains values that are multiples of 2.

Let us look at the size of *list3*. You can see that the size of *list2* is greater than *list1*. But the size of *list3* is the same.

```
>>> list2
[4, 6, 8, 10, 12]
```

Let us look at the size of *list1:*

```
>>> sys.getsizeof(list1)
104
```

Let us examine the size of *list2:*

```
>>> sys.getsizeof(list2)
128

>>> list3 = (each*2 for each in list1)
>>>
>>> next(list3)
4
>>> next(list3)
6
>>> next(list3)
8
>>> next(list3)
10
>>> next(list3)
12
>>> next(list3)
```

Traceback (most recent call last):

```
  File "<stdin>", line 1, in <module>
StopIteration
>>>
```

Let us compare the memory consumption of a generator expression and a *list* comprehension.

```
>>> import sys
>>> list1 = list(range(100000))
>>> list2 = [each*2 for each in list1]
>>> sys.getsizeof(list2)
824464
>>>
>>> list3 = (each*2 for each in list1)
>>> sys.getsizeof(list3)
120
```

You can see there is a huge difference in memory consumption. Let us compare the performance. Look at the following code and output.

```
import datetime
main_list = list(range(100000000))

t1 = datetime.datetime.now()
list1 = [each**2 for each in main_list]

for each in list1:

    each
t2 = datetime.datetime.now()
print (t2-t1)
```

I ran the code three times and checked the performance.

```
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python3 iter4.py
0:00:51.652933
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python3 iter4.py
0:00:50.144020
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python3 iter4.py
0:00:53.793114
```

When the *list* comprehension is converted to the generator, look at the following code:

```
import datetime
main_list = list(range(100000000))

t1 = datetime.datetime.now()
list1 = (each**2 for each in main_list)

for each in list1:
    each
t2 = datetime.datetime.now()
print (t2-t1)
```

The output is:

```
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python3 iter4.py
0:00:41.742375
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python3 iter4.py
0:00:40.739899
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python3 iter4.py
0:00:40.509761
```

The above result concludes that the generator's performance is better. END

By: Mohit

The author is a CEH and ECSA. He holds a master's degree in computer science engineering (ME) from Thapar University, Patiala. He has worked at IBM, Teramatrix and Sapient. He can be contacted at *mohitraj.cs@gmail.com*.